

PDET Municipality Boundaries Dataset Integration

Cuarta entrega

Eliana Katherine Cepeda

Juan Diego Gonzalez

Mateo Moreno

Noviembre 17, 2025

1. Resumen Ejecutivo

La Entrega 4, constituye una fase culminante con un objetivo central de este proyecto. Su propósito fundamental es cumplir con el requerimiento principal de la UPME: cuantificar de manera precisa el potencial solar de los territorios PDET. Esto se logra mediante la estimación del número total de edificios y el área agregada de tejados para cada municipio PDET, comparando los resultados de las dos fuentes de datos masivos: Microsoft Building Footprints y Google Open Buildings.

La metodología implementada en esta fase representa la evolución final de nuestro proceso, adoptando una estrategia de "Cargar-Luego-Analizar" (ELT). Tras los desafíos de memoria y tiempo de procesamiento identificados en la Entrega 3, se descartó el filtrado en Python. En su lugar, este flujo de trabajo presupone que la totalidad de los datos (aproximadamente 38 millones de registros) ya reside en colecciones optimizadas en MongoDB. El análisis reproducible se ejecuta mediante un script de Python que itera sobre los 170 polígonos municipales PDET. Para cada municipio, se delega el cómputo intensivo al servidor de la base de datos, ejecutando pipelines de agregación que utilizan el índice 2dsphere y el operador *\$geoIntersects* para filtrar, contar y sumar las áreas de manera eficiente.

De acuerdo con los requisitos del proyecto, los productos finales de este flujo de trabajo son dos. Primero, una estructura de datos tabulada (el archivo .csv generado), que sirve como el reporte cuantitativo principal. Segundo, un conjunto de visualizaciones geoespaciales (mapas coropléticos), que traducen estos datos tabulados a un formato visual, permitiendo identificar patrones y densidades del potencial solar sobre el mapa completo de Colombia. Además de la serie de pasos que se hizo.

2. Pasos Realizados

Primero se realizaron correcciones de la entrega anterior:

Antes	Corrección
- El ETL fue hecho de manera ineficiente, cargamos los datos filtrando y filtrándose por los que sean PDET y se subían a mongo. La memoria se reventaba o se demoraba tiempos exagerados en cargar, además de que todo se hacía en Python. - El script de Python consumía tanta RAM que el sistema lo "mataba" (se detenía a las 458k líneas en Microsoft y en Google se demoraba más de 10 horas).	"Cargar-Luego-Analizar" (ELT). Se cargaron los 6M de Microsoft y 32M de Google a la BD y el análisis se hizo <i>después</i> con consultas. - Se creó un script de carga robusto que insertaba los datos en lotes de 10k y forzaba la limpieza de memoria. - Se creó un script de Python que lee cada municipio PDET y le pide a MongoDB que sume las áreas y cuente los edificios usando <i>\$geoIntersects</i>.

- | | |
|---|--|
| <ul style="list-style-type: none">• Los datos estaban incompletos y, al usar un "molde unificado", no sabíamos a qué municipio pertenece cada edificio. | |
|---|--|

Ahora, en esta nueva entrega a comparación de la anterior cargamos todos los datos al mismo mongo, tanto los de Google como los de Microsoft de manera directa, siendo 6 millones Aprox de Microsoft y 32 millones Aprox de Google, además de los 170 edificios PDET.

El script necesita saber qué municipios debe analizar. Así que se ejecuta una consulta (`.find({})`) sobre la colección `municipios`. Todos los 170 documentos de los municipios PDET se cargan en una lista de Python llamada `municipios_pdet`. Esto se hace para tener la "lista de tareas" en la memoria: el script ahora sabe que debe repetir el análisis 170 veces, una por cada municipio. El script itera sobre la lista de 170 municipios (usando `tqdm` para mostrar una barra de progreso).

En cada una de las 170 iteraciones, el script extrae la geometría del municipio. Por ejemplo. el polígono de "Tibú". Esta geometría se usará como el filtro geoespacial para las consultas.

En lugar de traer millones de edificios a Python, el script le ordena a MongoDB que haga todo el trabajo. *Se hizo directamente en con python porque en Mongo Compass generaba un error.* Dentro de cada iteración del bucle, ejecuta dos "pipelines" de agregación:

- Pipeline (Google y Microsoft): Se define una plantilla de consulta que se ejecuta para ambas colecciones.
- Fase `$match` (Filtrar): Es el corazón de la consulta. Se usa el operador `$geoIntersects` para encontrar todos los edificios en la colección (Google o Microsoft) cuyo campo `centroide` caiga dentro de la geometría del municipio.
- Fase `$group` (Calcular): Inmediatamente después de encontrar los edificios (ej. 5,000 edificios en Tibú), la base de datos los agrupa en un solo documento y realiza dos operaciones matemáticas:
- "conteo": `{ "$sum": 1 }`: Suma 1 por cada edificio para obtener el conteo total.
- "area_total": `{ "$sum": "$area_m2" }`: Suma el `area_m2` de todos los edificios para obtener el área total.

Hacer el filtro (`$match`) y el cálculo (`$group`) dentro de la base de datos usando un pipeline es la optimización clave que permitió que el análisis terminará en minutos en lugar de días.

Nuevo Procesamiento datos Microsoft:

```
Iniciando el proceso de ETL para el archivo: D:\Javeriana\Bases de datos\proyecto\microsoft\Colombia.geojson1
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
--- ¡PROCESO COMPLETO PARA MICROSOFT! ---
Tiempo total: 18.26 minutos
Total de filas procesadas: 6083821
Total de edificios cargados en MongoDB: 6083821
```

Nuevo Procesamiento datos Google:

```
--- Procesando Archivo 2/9: 8e1_buildings.csv ---
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
Archivo '8e1_buildings.csv' completado.

--- Procesando Archivo 3/9: 8e3_buildings.csv ---
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
Archivo '8e3_buildings.csv' completado.

--- Procesando Archivo 4/9: 8e5_buildings.csv ---
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
Archivo '8e5_buildings.csv' completado.

--- Procesando Archivo 5/9: 8e7_buildings.csv ---
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
Archivo '8e7_buildings.csv' completado.

--- Procesando Archivo 6/9: 8e9_buildings.csv ---
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
Archivo '8e9_buildings.csv' completado.

--- Procesando Archivo 7/9: 8ef_buildings.csv ---
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
Archivo '8ef_buildings.csv' completado.

--- Procesando Archivo 8/9: 91d_buildings.csv ---
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
Archivo '91d_buildings.csv' completado.

--- Procesando Archivo 9/9: 91f_buildings.csv ---
Progreso (Chunks): 0 chunk [00:00, ? chunk/s]
Archivo '91f_buildings.csv' completado.

--- ¡PROCESO COMPLETO! ---
Tiempo total: 80.89 minutos
Total de filas CSV procesadas: 31521921
Total de edificios cargados en MongoDB: 31521921
```

Entonces, después de que MongoDB devuelve los resultados, el script los consolida en un nuevo diccionario añadiendo los datos del municipio (nombre, código DANE) y los resultados de las agregaciones de Google y Microsoft.

```
¡Conexión exitosa a MongoDB! BD: 'is394508_db'
Cargando los 170 municipios PDET en la memoria...
Se encontraron 170 municipios.
Iniciando análisis geoespacial (esto puede tardar)...
Procesando Municipios: 0%| | 0/170 [00:00<?, ?it/s]
--- ¡ANÁLISIS COMPLETO! ---
Tiempo total de procesamiento: 11.61 minutos
Mostrando los primeros 10 resultados:
municipio_nombre municipio_codigo_dane departamento google_conteo \
0 AMALFI 05031 ANTIOQUIA 2983
1 ANORÍ 05040 ANTIOQUIA 7652
2 APARTADÓ 05045 ANTIOQUIA 36337
3 BRICEÑO 05107 ANTIOQUIA 3565
4 CÁCERES 05120 ANTIOQUIA 12425
5 CAREPA 05147 ANTIOQUIA 16657
6 CAUCASIA 05154 ANTIOQUIA 30474
7 CHIGORODÓ 05172 ANTIOQUIA 18871
8 DABEIBA 05234 ANTIOQUIA 7696
9 EL BAGRE 05250 ANTIOQUIA 14768

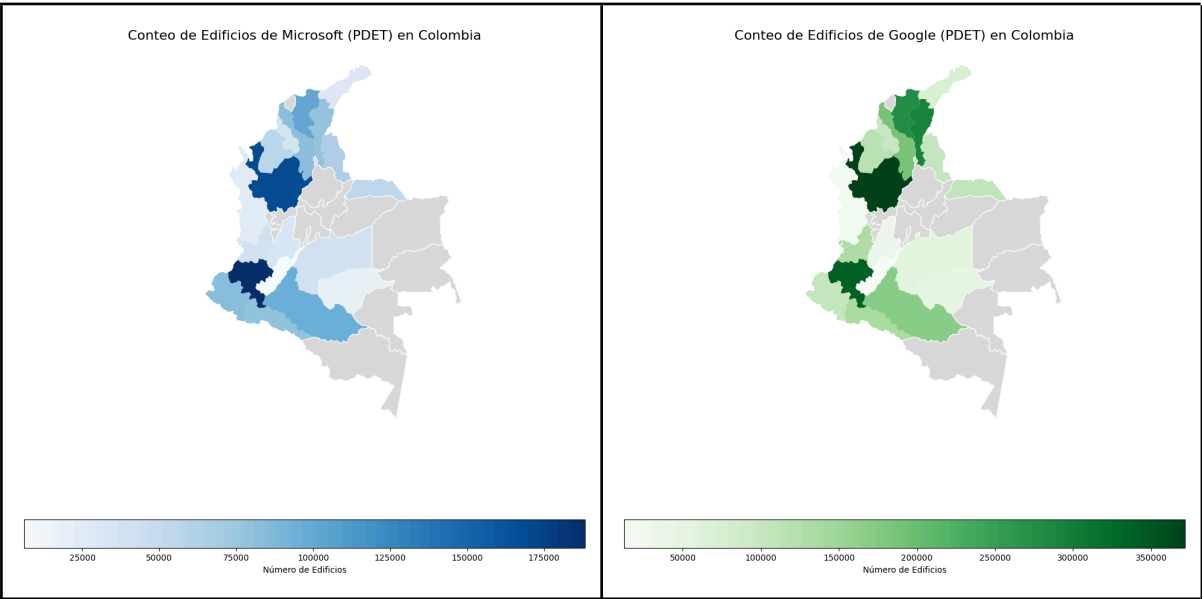
google_area_total_m2 microsoft_conteo microsoft_area_total_m2
0 1.856396e+05 5184 7.596969e+05
1 4.945226e+05 5318 5.422179e+05
2 3.606578e+06 9477 2.011083e+06
3 2.090405e+05 2113 2.082718e+05
4 9.725477e+05 6762 6.848477e+05
5 1.726620e+06 7039 1.123826e+06
6 2.771754e+06 10208 1.737321e+06
7 1.883086e+06 7078 1.680870e+06
8 5.061714e+05 4766 4.949283e+05
9 1.314787e+06 7134 7.486756e+05
```

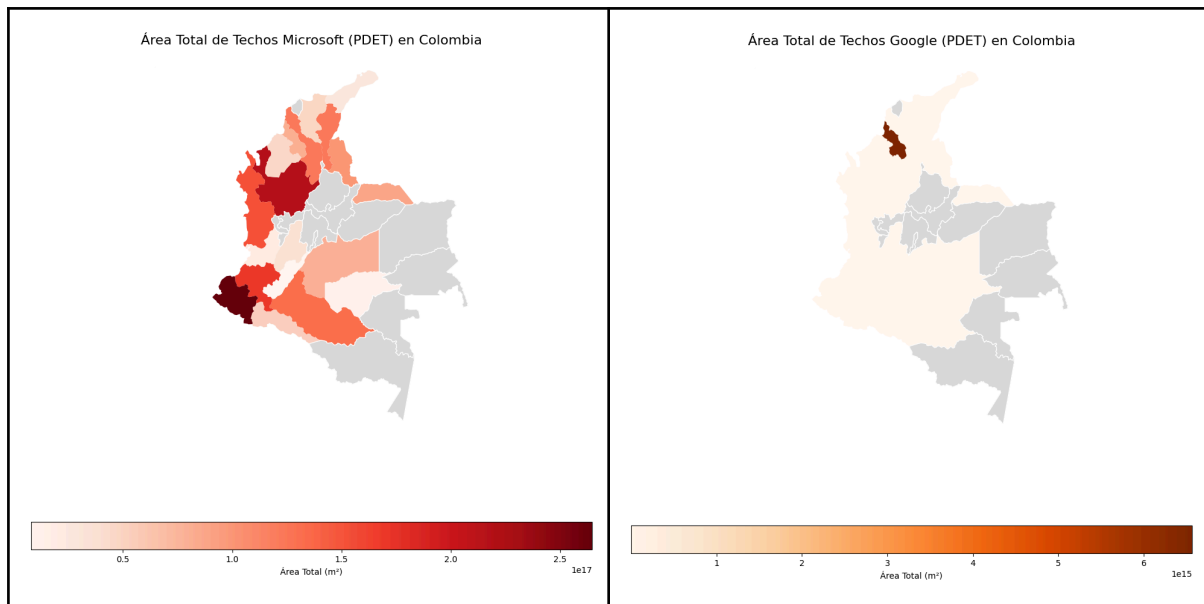
Resultados guardados en 'reporte_final_entrega_4.csv'

Además que maneja los casos donde no se encuentran edificios (ej. `if stats_google else 0`) para evitar errores y rellenar con ceros. Termina de ejecutar el Script de Python y se guarda todo en un excel con los datos agrupados por municipio.

	A	B	C	D	E	F	G
1	municipio_nombre	municipio_codigo_dane	departamento	google_conteo	google_area_total_m2	microsoft_conteo	microsoft_area_total_m2
2	AMALFI	5031	ANTIOQUIA	2983	1856396093	5184	7,59697E+15
3	ANORÍ	5040	ANTIOQUIA	7652	4945226483	5318	5,42218E+15
4	APARTADÓ	5045	ANTIOQUIA	36337	3606578248	9477	2,01108E+15
5	BRICEÑO	5107	ANTIOQUIA	3565	2090404883	2113	2,08272E+15
6	CÁCERES	5120	ANTIOQUIA	12425	9725476676	6762	6,84848E+15
7	CAREPA	5147	ANTIOQUIA	16657	17266204024	7039	1,12383E+16
8	CAUCASIA	5154	ANTIOQUIA	30474	2771754468	10208	1,73732E+16
9	CHIGORODÓ	5172	ANTIOQUIA	18871	18830855872	7078	1,68087E+16
10	DABEIBA	5234	ANTIOQUIA	7696	5061714122	4766	4,94928E+15
11	EL BAGRE	5250	ANTIOQUIA	14768	13147871012	7134	7,48676E+15
12	ITUANGO	5361	ANTIOQUIA	9063	627500743	5776	6,00723E+15
13	MURINDÓ	5475	ANTIOQUIA	0	0	927	8,59907E+15
14	MUTATÁ	5480	ANTIOQUIA	5537	4693866149	3118	3,41967E+15
15	NECOCLÍ	5490	ANTIOQUIA	32173	23441543329	12879	1,21782E+16
16	NECHÍ	5495	ANTIOQUIA	3492	2319781305	2852	2,5068E+16
17	REMEDIOS	5604	ANTIOQUIA	13248	9630312446	8345	9,26883E+15
18	SAN PEDRO DE URABÁ	5665	ANTIOQUIA	22034	16447550511	9947	1,14195E+15
19	SEGOVIA	5736	ANTIOQUIA	16158	119618533	7428	9,13841E+15

3. Visualizaciones Mapas:





4. Conclusiones

- Se mejoró la entrega 3 y se buscó una alternativa mucho mas optima y funcional. La estrategia ELT superó al ETL: El intento inicial de "Filtrar-Luego-Cargar" (ETL) en Python fracasó debido a las severas limitaciones de memoria RAM de la máquina local. La solución exitosa fue invertir el proceso a "Cargar-Luego-Analizar" (ELT), cargando primero los 38 millones de registros y moviendo todo el trabajo pesado a la base de datos.
- Se demostró que las operaciones geoespaciales (`$geoIntersects`) ejecutadas por el motor de MongoDB, usando un índice `2dsphere`, son exponencialmente más rápidas que intentar el mismo filtro en Python. La base de datos es la herramienta idónea para el cómputo geoespacial a gran escala.
- Fue necesario implementar soluciones para manejar geometrías inválidas (ej. `sparse=True` en Microsoft) y asegurar la precisión de los cálculos (ej. reproyectar a coordenadas planas antes de calcular áreas y centroides).
- A pesar de los desafíos técnicos, el flujo de análisis final de la Entrega 4 fue reproducible y exitoso. Se generó el reporte tabulado (.csv/.xlsx) y las visualizaciones (mapas) que cuantifican el número de edificios y el área total de techos por municipio, cumpliendo con el objetivo principal de la UPME.